

RTL Implementation of RISC-V Out-of-Order Processor Using Tomasulo's Algorithm

Antonio Anzora Jr.

Department of Electrical and Computer Engineering,
California State University, Northridge
May 1st, 2026

Abstract—Out-of-Order is a fundamental technique in modern high-performance processors that improves instruction-level parallelism by allowing instructions to execute based on data availability rather than program order. Tomasulo's algorithm was deployed in IBM360/91 floating point unit it's similar to the score boarding technique. This paper presents a cycle accurate Register-Transfer Level (RTL) implementation of a RISC-V out-of-order processor using Tomasulo's Algorithm, written entirely in VHDL. The design includes key components such as Register Alias Table (RAT) for register renaming, reservation stations (RS) for dynamic instruction scheduling, a Common Data Bus (CDB) for result broadcasting, and a ReOrder Buffer (ROB) for in-order commitment. The processor executes a subset of the RISC-V RV32I instruction set and is verified through detailed waveform simulations demonstrating correct out-of-order execution, elimination of register hazards, and in-order commit semantics. Simulation results show the processor successfully executes multiple instructions in parallel with proper handling of data dependencies. This implementation serves as both an educational tool for understanding modern CPU microarchitecture and a demonstration of complex hardware design at the RTL level. The rest of the paper is organized as Section I provides Introduction, Section II background on Tomasulo's Algorithm, Section III describes System Architecture, Section IV details the VHDL implementation, and Section V presents simulation results.

Index Terms—Out-of-Order execution, Tomasulo's Algorithm, Register Renaming, Dynamic Instruction Scheduling, RISC-V, VHDL

I. INTRODUCTION

MODERN high-performance processors achieve instruction-level parallelism (ILP) through out-of-order execution, allowing instructions to execute based on data availability rather than program order. While three primary approaches exist for handling instruction issue and completion in-order completion, in-order completion/out-of-order completion, and out-of-order issue/out-of-order completion, where the last approach offers the shortest cycle time by maximizing parallelism. However, this requires register renaming to eliminate false data dependencies (WAR and WAW hazards) while preserving true data dependencies.

Tomasulo's Algorithm, originally implemented in the IBM System/360/91 solves this through dynamic instruction and scheduling and register renaming via a ReOrder Buffer and Reservation Stations. Compared to earlier approaches like score boarding, Tomasulo requires less tracking because each function unit maintains only local information about operand

readiness, reducing complexity while improving cycle time.

This paper presents a cycle-accurate RTL implementation of a RISC-V out-of-order process using Tomasulo's Algorithm in VHDL. By implementing this at the register-transfer level, we looked into the actual control flow and timing behavior of modern CPU microarchitecture without a physical hardware or FPGA board. The implementation includes eight key components: a Register File, Register Alias Table (RAT) for renaming, Instruction Queue, Reservation Stations for dynamic scheduling, function units (ALU and MEM), Common Data Bus for result broadcasting, and Reorder buffer for in-order commit. Simulation results verify correct out-of-order execution with proper dependency handling in-order commit semantics.

II. BACKGROUND AND RELATED WORK

Instruction-Level Parallelism (ILP) improves processor performance by executing multiple instructions per cycle, reducing cycles per instruction (CPI). Two primary approaches achieve this superscalar processors and Very Long Instruction Word (VLIW) processors. Superscalar processors employ one of two scheduling strategies, static (in-order issue, in-order completion) or dynamic (out-of-order issue, out-of-order completion). Static scheduling relies on the compiler to identify parallelism at runtime by reordering instructions based on data availability. VLIW processors use static scheduling, where the compiler explicitly packages multiple independent instructions into long instruction words.

Tomasulo's Algorithm, first implemented in the IBM System/360/91, is a dynamic scheduling technique that executes instruction in three stages Issue, Execute, Write Result. Issue instructions are fetched from an instruction queue (FIFO) and issued to Reservation Stations (RS) if space is available. If operand values are ready, they are forwarded immediately, and if not, the instruction stalls and waits for operands. Execute is where all operands become available (either immediately or from Common Data Bus), the instruction executes in a functional unit. Write Result are where results are broadcast on the Common Data Bus (CDB) and snooped by all Reservation Stations waiting for that result. Stores must also wait for both the address and data values.

Register Renaming removes false data dependencies (WAR and WAW hazards) by mapping architectural registers to physical registers dynamically. A Register Alias Table (RAT) tracks which ROB entry holds the most recent value for each register. Reservation Stations buffer operands as they become available on the CDB. This allows instructions to execute out

of program while the processor commits them in order, ensuring correct results.

III. SYSTEM ARCHITECTURE

A. Component Functions

The processor consists of eight components such as Register File 32, Register Alias Table (RAT), Instruction Queue (IQ), Reservation Stations (RS), ALU, MEM Unit, Control Data Bus (CDB), and Reorder Buffer (ROB).

Register.

- Register File: Stores the 32 architectural registers (x0 – x31) and provides operand values to the issue stage.
- Register Alias Table (RAT): Maps registers to ROB entries, tracking which in-flight instruction will produce each register's value. This enables register renaming and eliminates false dependencies (WAR and WAW hazards).
- Instruction Queue (IQ): An 8-entry FIFO that buffers fetched instructions and supplies them to the issue unit one per cycle. When full, it signals the fetch stage to stall.
- Reservation Stations (RS): Four entries that buffer instructions waiting operands. RS accepts issued instructions, snoops the CDB for missing operands, and broadcasts ready instructions to function units.
- ALU and MEM: Functional units that execute arithmetic/logic and load/store operations respectively.
- Common Data Bus (CDB): Broadcasts results from functional units. All RS entries snoop the CDB to detect when operands become available.
- Reorder Buffer (ROB): An 8-entry circular buffer that tracks in-flight instructions. ROB accepts new instructions, marks them done when results appear on CDB, and commits them in program order to the register file.

B. Data Flow and Execution

Instruction flow through three stages per cycle: Issue, Execute, Write Result.

- Issue: IQ supplies an instruction to RS with its operands from RF or RAT tags. If operands are not ready, RS stores the tag and waits.
- Execute: When all operands are available snooped from CDB or immediately RS broadcasts the instruction to the appropriate functional unit.
- Write Result: The functional unit broadcasts its result on CDB. All RS entries snoop and capture results from waiting instructions. ROB marks the instruction done, and if it is the head of the ROB, commits the result to the register file.

Without RAT, ROB, and CDB, the processor would suffer WAW/WAR hazards causing stalls. RAT eliminates false dependencies, ROB enables in-order commitment, and CDB coordinates result distribution.

IV. IMPLEMENTATION DETAILS

A. Design Approach

The implementation began with some VHDL exercises (D-flip-flop, FIFO, and basic register file) to get familiar with hardware description language syntax. Following this, the eight core components were designed and verified individually: Register File (RF), Register Alias Table (RAT), Instruction Queue (IQ), Reservation Stations (RS), ALU, Memory Unit, CDB, and ROB.

B. Component Organization

Each component was implemented in VHDL separately with its own entity and testbench allowing independent verification before integration. This approach was to ensure that each component was met with its specifications. All components were then instantiated in a top-level entity (TOP.vhd) that handles interconnection.

C. Control Logic

A control unit issues one instruction every two cycles, checking three conditions:

1. Instruction Queue is not empty
2. Reservation Stations are not full
3. Reorder Buffer is not full

This approach prevents deadlock and ensures a smooth pipeline flow.

V. RESULTS / SIMULATION

The complete processor consists of eight interconnected components. To simplify the complex schematic, its flow is: Instruction Queue (IQ) → Register Alias Table (RAT) → Reservation Stations (RS) → Functional Units (ALU/MEM) → Common Data Bus (CDB) → Reorder Buffer (ROB) → Register File (RF). Instructions flow through this pipeline, with the RAT performing register renaming, RS buffering operands and scheduling dynamically, functional units executing when ready, CDB broadcasting results, and ROB committing in order.

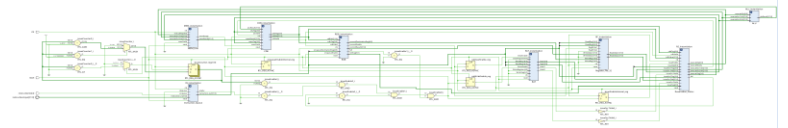


Figure 1. RISC-V Out-of-Order Processor Block Diagram

The waveform demonstrates dual-port simultaneous writes. At 40-60ns, register x1 receives 0xAAA_AAAA on write port 1. At 60-80ns, register x2 receives a different value on write port 2. The test then verifies parallelisms by writing to x3 and x4 simultaneously with different values. Finally, at 100-120ns, both x1 and x2 are read, confirming storage and retrieval. This demonstrates the issue instructions with two source operands.



Figure 2. Register_File_32: Dual-port write, x0 protection
The waveform shows the Register Alias Table tagging registers with ROB entries. At 40-60ns, register x1 is written with tag

0011 (ROB entry 3). At 60-80ns, register x2 is written with tag 0101 (ROB entry 5). From 80-100ns, reads on x1 and x2 return their corresponding tags, which shows that the ROB entries hold the most recent values. This shows register renaming are mapped to in-flight ROB entries, enabling out-of-order execution while tracking value dependencies.



Figure 3. RAT: Register renaming and tag tracking

The waveform demonstrates two key behaviors. First (40-60ns), an ADD instruction with both operands ready (src1=10, src2=20) issues to RS and executes immediately. Second (60-100ns), a dependent instruction issues with one operand ready (src1=5) and one waiting for ROB#2. When CDB broadcasts ROB#2's result at 80ns, RS snoops this value and completes the missing operand. The instruction then becomes ready to execute. This shows that the instructions wait only for their operands, not for prior instructions to complete.



Figure 4. Reservation_Status: instruction issuing and CDB snooping

The waveform demonstrates three behaviors. First (40-80ns), three instructions enqueue to ROB entries 0, 1, and 2. Next (80-140ns), CDB broadcasts complete each instruction's result sequentially (ROB#0: 0x10, ROB#1: 0x20, ROB#2: 0x30), and each commit in order. Finally (160-240ns), eight more instructions fill the ROB. At 240ns, the full signal asserts, and a ninth enqueue attempt is ignored. This shows in-order commitment of execution order; instructions retire from the ROB in program order.

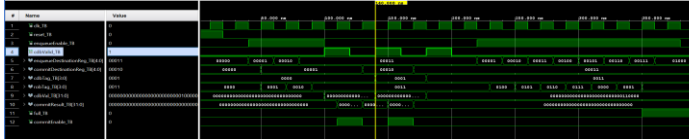


Figure 5. ROB: in-order commit

The waveform shows the complete pipeline executing three instructions (ADDI, LW, ADD) with out-of-order execution and in-order commitment. Instructions enter via instructionInput_TB, are proceeded through RAT and RS, execute when operands are ready (rsExecuteReady pulses), and broadcast results on CDB (cdbValid, cdbVal pulses). Finally, the ROB commits instructions sequentially (commitEnable pulses, commitResult shows values). Noticing the cdbValid pulses at different times (out-of-order execution), but commitEnable pulses sequentially (in-order commitment) that's the Tomasulo's Algorithm in action.



Figure 6. Full OOO pipeline execution

VI. REFERENCES

- [1] Dr. Mirzaei Shahnam, "ECE 622: Computer Systems Architecture Course Notes," California State University, Northridge, Spring 2026.
- [2] Wikipedia, "Tomasulo's Algorithm," [Online]. Available: https://en.wikipedia.org/wiki/Tomasulo%27s_algorithm. [Accessed: May 2026].